

Since you haven't specified the exact backend stack (e.g., Node.js, Java Spring, Go, Python), I have designed this architecture using **industry-standard patterns for high-availability, security-first banking systems**.

1. Deployment Architecture

- **Pattern:** Multi-AZ (Availability Zone) Microservices on Kubernetes.
- **Ingress:** Cloud Load Balancer (WAF-enabled) -> Ingress Controller (Nginx) -> Service Mesh (Istio for mTLS).
- **Security:** Strict network segmentation (Private Subnets). Backend services never have public IP addresses.

2. Docker

- **Strategy:** Multi-stage builds to ensure small image footprints and security.
- **Base Image:** Use Distroless or Alpine Linux images (minimal attack surface).
- **Security:** Scan images in the CI pipeline (Trivy) and sign them (Cosign).

```
# Example: Multi-stage build
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

FROM gcr.io/distroless/nodejs20-debian12
COPY --from=builder /app/dist /app/dist
USER nonroot
CMD ["/app/dist/main.js"]
```

3. CI/CD (GitHub Actions / GitLab CI)

- **Pipeline Stages:**

1. **Lint/Test:** Unit tests + Integration tests.
2. **Security Scan:** SAST (SonarQube) + Dependency scanning.

3. **Build/Push:** Build Docker image and push to private Container Registry (ECR/GCR).

4. **Deploy:** Helm chart update to Kubernetes (ArgoCD for GitOps).

- **Strategy:** Blue-Green or Canary deployments to ensure zero downtime.

4. Hosting

- **Cloud Provider:** AWS (EKS - Elastic Kubernetes Service).
- **Network:** VPC with private subnets for app servers, public subnets for Load Balancers only.

5. Database Hosting

- **Primary:** AWS RDS (PostgreSQL) Multi-AZ.
- **Encryption:** AES-256 at-rest; TLS 1.3 in-transit.
- **Access:** Private Subnet only; accessible only by the Backend Security Group.
- **Audit:** Enable Database Activity Streams for regulatory compliance (PCI-DSS/GDPR).

6. Environment Variables

- **Management:** Do not use plain text. Use **AWS Secrets Manager** or **HashiCorp Vault**.
- **Injection:** Use a Kubernetes Secret Store CSI driver to mount secrets as files or environment variables at runtime.

7. Monitoring

- **Stack:** Prometheus (Metrics) + Grafana (Visualization).
- **Tracing:** Jaeger or OpenTelemetry (required for tracking financial transactions across microservices).
- **Alerting:** Alertmanager integrated with PagerDuty for critical P0/P1 issues.

8. Logging

- **Stack:** EFK (Elasticsearch, Fluentd, Kibana) or Datadog Logs.
- **Requirement:** Banking systems require immutable logs. Use S3 Object Lock for long-term log retention to comply with audit requirements.

9. Scaling

- **Horizontal Pod Autoscaler (HPA):** Scales based on CPU/Memory and custom metrics (e.g., Request Queue depth).
- **Cluster Autoscaler:** Scales the underlying EC2 node count when pods exceed current cluster capacity.

10. Backup Strategy

- **Database:**

* Continuous PITR (Point-in-Time Recovery) enabled (Retention: 35 days).

* Weekly snapshots moved to S3 (Cold Storage/Glacier) for 7-year compliance.

- **Disaster Recovery:** Cross-region replication of the database and automated infra-as-code (Terraform) recreation capability.

Architect's Advice for Banking Systems:

1. **Immutability:** Ensure your infrastructure is "Infrastructure as Code" (Terraform/OpenTofu). Never manual changes.
2. **Zero Trust:** Implement mTLS between all microservices. Assume the network is compromised.
3. **Idempotency:** Every financial API must be idempotent. Implement `Idempotency-Key` headers for all transaction requests to prevent double-charging during network retries.